

PES UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ACADEMIC YEAR 2025-2026 | FIFTH SEMESTER B.TECH

SOFTWARE TEST PLAN
(STP Specification)
PROJECT: SPRINTFLOW – PROJECT MANAGEMENT SYSTEM
(Workflow, Visibility, Estimation, and Kanban)

SUBMITTED BY: TEAM 11 (SECTION 5C)

Student Name	USN	Role	Assigned Deliverable
Darshan P Pawar	PES2UG24CS143	Team Lead & Architect	Architecture, ADRs, Component Diagram
Dhatri Shivaprasad	PES2UG24CS158	Frontend & UI/UX Lead	Design Overview, UX Design, Sequence Diagrams
Chandan Kumar K	PES2UG24CS128	Backend & Database Architect	API Design, Data Stores, Security Architecture
Gurubelli Yekambar Eshwar Rao	PES2UG24CS173	Full-Stack & QA/DevOps Lead	Test Strategy, Metrics, CI/CD Pipeline

Version: 1.0 (Final Submission) | Date: 15-09-2026 | Status: Approved
Repository: <https://github.com/Darshanpawar7/SE-SRS-2>

Version: 1.0

Authors: Team 11 (Section 5C, PES University)

Date: 15-09-2026

Status: Final Submission / Approved

Revision History

Version	Date	Author	Change Summary
1.0	15-09-2026	Team 11	Complete STP with test strategy, 18 FR and 6 NFR test mappings, security validation, RTM, and metrics framework

1. Introduction

Purpose: This document defines the Software Test Plan (STP) for SprintFlow v1.0, an Agile Project Management System. It establishes the objectives, scope, strategy, resources, schedule, and responsibilities for all testing activities, ensuring that the delivered system satisfies the functional and non-functional requirements specified in the Software Requirements Specification (SRS v1.0).

Scope: Testing covers all SprintFlow application features including user authentication, Kanban board workflow, task management, sprint lifecycle, Fibonacci estimation, executive dashboard, burndown chart visualization, Supabase cloud database synchronization, offline LocalStorage persistence, and the CI/CD pipeline. Excluded from testing scope: Supabase internal infrastructure, browser rendering engine internals, third-party CDN operations, and operating system-level behaviors.

References:

- SprintFlow SRS v1.0 ([SE-SRS Repository])(<https://github.com/Darshanpawar7/SE-SRS>)
- SprintFlow SAD v1.0 ([SE-SRS-2 Repository])(<https://github.com/Darshanpawar7/SE-SRS-2>)
- Requirements Traceability Matrix (SRS Section 8)
- Database Schema: `database/schema.sql`
- CI/CD Workflow: `.github/workflows/ci.yml`
- IEEE 829: Standard for Software and System Test Documentation

Definitions:

Term	Definition
STP	Software Test Plan
SRS	Software Requirements Specification
SAD	Software Architecture and Design Specification
RTM	Requirements Traceability Matrix
FR	Functional Requirement
NFR	Non-Functional Requirement
UAT	User Acceptance Testing
CI/CD	Continuous Integration / Continuous Deployment
RLS	Row Level Security
XSS	Cross-Site Scripting
DnD	Drag and Drop

2. Test Items

The following SprintFlow modules constitute the test items for this plan:

Module	Source Path	Description
Authentication Module	src/context/AuthContext.jsx	User login, profile switching, session management
Kanban Board Module	src/components/kanban/KanbanBoard.jsx	5-column workflow display, drag-and-drop transitions, 1-click advancement
Task Management Module	src/components/kanban/TaskModal.jsx	Task creation, editing, priority assignment, story point allocation
Sprint Management Module	src/components/sprint/SprintManager.jsx	Sprint creation, goal definition, task assignment, lifecycle tracking
Executive Dashboard Module	src/components/dashboard/DashboardView.jsx	KPI metrics, velocity calculation, workload distribution, SVG burndown chart
Database Service Module	src/services/supabaseClient.js	Dual-mode data persistence (Supabase cloud sync and LocalStorage offline fallback)
Settings Module	src/components/settings/SettingsModal.jsx	Supabase connection configuration (Project URL, Anon Key)

3. Features to be Tested

Features mapped to SRS requirement IDs:

3.1 Functional Requirements

Req ID	Feature Description	Test Priority
PMS-F-001	User authentication and profile role switching	High
PMS-F-002	Create new task cards with metadata (title, description, priority, assignee)	High
PMS-F-003	Display 5-column Kanban board (Backlog, To Do, In Progress, Code Review, Done)	High
PMS-F-004	Drag-and-drop task stage transitions between columns	High
PMS-F-005	1-click task stage advancement button	Medium
PMS-F-006	Fibonacci story point assignment (1, 2, 3, 5, 8, 13, 21)	High
PMS-F-007	Sprint creation with name, goal, start date, and end date	High
PMS-F-008	Sprint capacity calculation (sum of assigned story points)	Medium
PMS-F-009	SVG burndown chart visualization (ideal vs actual trajectory)	Medium
PMS-F-010	Executive KPI dashboard with progress percentage and velocity	Medium
PMS-F-011	Task filtering by status, priority, and assignee	Medium
PMS-F-012	Audit trail logging for all data mutations	High
PMS-F-013	Supabase PostgreSQL cloud database synchronization	High
PMS-F-014	Offline LocalStorage persistence (zero-config mode)	High
PMS-F-015	Database settings configuration UI (Project URL, Anon Key)	Low
PMS-F-016	Task priority assignment (Low, Medium, High, Critical)	Medium
PMS-F-017	Workload balance visualization per team member	Low
PMS-F-018	In-app documentation viewer	Low

3.2 Non-Functional Requirements

Req ID	Feature Description	Test Priority
PMS-NF-001	UI state transitions complete within 50ms	High
PMS-NF-002	System availability target of 99.9%	Medium
PMS-NF-003	Cross-browser compatibility (Chrome, Firefox, Edge)	High
PMS-NF-004	Data encryption in transit via TLS 1.3	High
PMS-NF-005	Support 100+ concurrent task records	Medium

Req ID	Feature Description	Test Priority
PMS-NF-006	Recovery from network disconnection without data loss	High

4. Features Not to be Tested

- Supabase internal PostgreSQL engine behavior and query optimizer internals
- Browser rendering engine CSS layout calculations
- Third-party CDN availability and content delivery performance
- Operating system-level network stack behavior
- Node.js runtime internals and V8 engine optimizations
- npm package registry availability during dependency installation

5. Test Approach / Strategy

5.1 Test Levels

Level	Description	Tools	Scope
Unit Testing	Isolated testing of individual functions and utility modules	Vitest	Estimation logic, workflow state transitions, data validation
Integration Testing	Testing interactions between components and the data service layer	Vitest, manual	AuthContext-SupabaseClient, ProjectContext-KanbanBoard, SprintManager-Estimation
System Testing	End-to-end testing of complete user workflows	Manual browser testing	Full authentication-to-burndown flow, offline-to-online sync
Acceptance Testing (UAT)	Validation against SRS acceptance criteria by evaluators	Manual demonstration	All 18 FRs verified against acceptance criteria

5.2 Test Types

Type	Purpose	Applicable Requirements
Functional Testing	Verify correct behavior of all system features	PMS-F-001 through PMS-F-018
Regression Testing	Ensure new changes do not break existing functionality	All requirements after code modifications
Performance Testing	Validate response time and throughput targets	PMS-NF-001, PMS-NF-005
Usability Testing	Assess UI clarity, navigation flow, and accessibility	PMS-NF-003, UX requirements
Security Testing	Validate authentication, authorization, and data protection	PMS-NF-004, PMS-SR-001 through PMS-SR-005

5.3 Entry and Exit Criteria

Entry Criteria:

- Stable build delivered (npm run build completes without errors)
- All unit tests passing (npm test reports 0 failures)
- Test environment ready (Node.js 18+, browser available)
- Test data available (seed data in mockData.js)
- SRS and SAD documents reviewed and approved

Exit Criteria:

- 100% of planned test cases executed
- 0 critical or high-severity defects remain open
- All 18 functional requirements verified against acceptance criteria
- All 6 non-functional requirements measured and within threshold

- Requirements Traceability Matrix shows complete coverage
- Test Summary Report reviewed and approved

5.4 Security Validation

Security Test	Validation Method	Pass Criteria
XSS Input Sanitization	Inject script tags and event handlers into all text input fields (task title, description, sprint goal)	No script execution; all input rendered as plain text via React JSX auto-escaping
Row Level Security (RLS)	Attempt to query tasks belonging to a different project using direct Supabase API calls	403 Forbidden response; zero unauthorized rows returned
TLS 1.3 Enforcement	Inspect network traffic using browser DevTools during Supabase API communication	All requests use HTTPS with TLS 1.3; no plaintext HTTP requests observed
Session Token Handling	Examine browser storage for JWT tokens after authentication	Tokens stored securely; expired tokens rejected on subsequent API calls
API Key Exposure	Review client-side JavaScript bundle for hardcoded secrets	Only anon (public) key present; no service_role or secret keys in client bundle
Input Validation Fuzzing	Submit malformed data (empty strings, excessively long strings, special characters) to all form fields	Application handles gracefully with validation error messages; no crashes or unhandled exceptions

6. Test Environment

6.1 Hardware

Component	Specification
Development Machine	Windows 10/11, 8GB+ RAM, modern x86-64 processor
Display	Minimum 1024x768 resolution for responsive layout testing
Network	Broadband internet connection for Supabase cloud testing; disconnection simulation for offline testing

6.2 Software

Component	Version	Purpose
Node.js	18.0.0+	JavaScript runtime for build and test execution
npm	9.0.0+	Package manager for dependency management
Vite	5.x	Development server and production bundler
Vitest	2.x	Unit test framework and runner
Google Chrome	Latest	Primary testing browser
Mozilla Firefox	Latest	Cross-browser compatibility testing
Microsoft Edge	Latest	Cross-browser compatibility testing
Git	Latest	Version control
GitHub Actions	Latest	CI/CD pipeline execution

6.3 Test Data

- Pre-configured seed data in `src/services/mockData.js` providing:
 - 4 user profiles (one per team member with assigned roles)
 - 12 sample tasks distributed across all 5 Kanban columns
 - 2 sample sprints (1 active, 1 completed) with assigned tasks
 - Audit log entries demonstrating mutation tracking

7. Test Schedule

Milestone	Planned Date	Status
Test Plan finalization	15-09-2026	Complete
Test case design and review	16-09-2026	Complete
Test environment setup and validation	16-09-2026	Complete
Unit test execution (Vitest automated suite)	16-09-2026	Complete
Integration test execution	17-09-2026	Complete
System test execution (end-to-end workflows)	17-09-2026	Complete
Security validation execution	18-09-2026	Complete
Performance test execution	18-09-2026	Complete
User Acceptance Testing (UAT)	19-09-2026	Complete
Test Summary Report generation	19-09-2026	Complete

8. Test Deliverables

Deliverable	Description
Software Test Plan (this document)	Defines scope, strategy, schedule, and responsibilities for all testing activities
Test Cases	Detailed test case specifications mapping to SRS requirement IDs
Automated Test Scripts	Vitest unit test files (<code>src/tests/estimation.test.js</code> , <code>src/tests/workflow.test.js</code>)
Test Data	Seed data configuration in <code>src/services/mockData.js</code>
CI/CD Test Execution Logs	GitHub Actions workflow run logs capturing build, test, and lint results
Test Summary Report	Final summary of test execution results, defect statistics, and coverage metrics

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	GURUBELLI YEKAMBAR ESHWAR RAO (PES2UG24CS173)	Prepare test plan, coordinate test execution, compile test summary report
Test Engineer (Frontend)	DHATRI SHIVAPRASAD (PES2UG24CS158)	Design and execute UI/UX test cases, usability testing, cross-browser validation
Test Engineer (Backend)	CHANDAN KUMAR K (PES2UG24CS128)	Design and execute data service test cases, database integration testing, security testing
Test Architect	Darshan P Pawar (PES2UG24CS143)	Define test strategy, review test cases, approve test results, CI/CD pipeline oversight

10. Risks and Mitigation

Risk	Probability	Impact	Mitigation
Supabase sandbox unavailability during cloud integration testing	Medium	High	Maintain parallel test execution using LocalStorage offline mode; schedule cloud tests during off-peak hours
Cross-browser rendering inconsistencies affecting test reliability	Medium	Medium	Standardize testing on Chrome as primary browser; document browser-specific deviations as known issues
Insufficient test data for edge case coverage	Low	Medium	Extend seed data in <code>mockData.js</code> to include boundary conditions (empty sprints, maximum story points, 100+ tasks)
CI/CD pipeline timeout on GitHub Actions free tier	Low	Low	Optimize test execution time; split long-running tests into parallel jobs
Network instability during offline-to-online sync testing	Medium	Medium	Use browser DevTools network throttling to simulate controlled disconnection scenarios

Risk	Probability	Impact	Mitigation
Dependency version conflicts after npm install	Low	Medium	Lock dependency versions in package-lock.json; run npm ci in CI/CD for deterministic builds

11. Assumptions and Dependencies

Assumptions:

- Node.js 18+ and npm 9+ are available on all testing machines
- Supabase cloud sandbox will be accessible for cloud integration testing
- Seed data in mockData.js provides sufficient coverage for functional testing
- Browser DevTools are available for network simulation and performance profiling
- GitHub Actions runners are available for CI/CD pipeline execution
- All team members have Git and GitHub access for branch-based test artifact management

Dependencies:

- SRS v1.0 document must be finalized before test case design (satisfied)
- SAD v1.0 document must be finalized before integration test design (satisfied)
- Application codebase must compile and pass unit tests before system testing
- Supabase project must be provisioned for cloud database integration testing
- GitHub repository must be configured with CI/CD workflow for automated test execution

12. Suspension and Resumption Criteria

Suspend testing if:

- Production build fails (npm run build returns non-zero exit code)
- More than 30% of unit tests fail, indicating systemic instability
- Test environment is unavailable for more than 4 hours
- A critical defect is discovered that blocks further test execution
- Supabase cloud service experiences a documented outage affecting all regions

Resume testing when:

- Build failures are resolved and verified by a clean npm run build
- Blocking defects are fixed, verified, and the fix is merged to the test branch
- Test environment is restored and validated
- Supabase service status returns to operational

13. Test Case Management and Traceability

The Requirements Traceability Matrix ensures bidirectional mapping between SRS requirements and test cases. Each requirement ID is traced to one or more test cases with defined pass/fail criteria:

13.1 Functional Requirements Traceability

Req ID	Requirement	Test Case ID	Test Description	Expected Result
PMS-F-001	User authentication	TC-AUTH-01	Enter valid credentials and verify login	User profile displayed in Navbar; authenticated state persisted
PMS-F-002	Task creation	TC-TASK-01	Create task via TaskModal with all fields populated	Task card appears in Backlog column with correct metadata
PMS-F-003	Kanban board display	TC-KAN-01	Load application and verify 5-column layout	All 5 columns rendered with correct headers and task cards
PMS-F-004	Drag-and-drop transitions	TC-KAN-02	Drag task from Backlog to In Progress	Task moves to target column; status updated in data store

Req ID	Requirement	Test Case ID	Test Description	Expected Result
PMS-F-005	1-click advancement	TC-KAN-03	Click advance button on a To Do task	Task moves to In Progress column
PMS-F-006	Story point assignment	TC-EST-01	Assign story points (1,2,3,5,8,13,21) to a task	Points saved and reflected in sprint capacity
PMS-F-007	Sprint creation	TC-SPR-01	Create sprint with name, goal, and date range	Sprint appears in SprintManager with correct metadata
PMS-F-008	Capacity calculation	TC-EST-02	Assign 3 tasks (3+5+8 points) to a sprint	Sprint capacity displays 16 story points
PMS-F-009	Burndown chart	TC-BURN-01	Complete tasks within an active sprint	SVG chart updates showing actual burn vs ideal line
PMS-F-010	KPI dashboard	TC-DASH-01	View dashboard with active sprint data	Progress percentage, velocity, and workload metrics displayed
PMS-F-011	Task filtering	TC-FILT-01	Apply priority filter (High) on Kanban board	Only High-priority tasks visible; others hidden
PMS-F-012	Audit logging	TC-LOG-01	Create, update, and delete a task	Audit log contains 3 entries with correct timestamps and actions
PMS-F-013	Supabase sync	TC-SUPA-01	Configure Supabase credentials and create a task	Task persisted in PostgreSQL; retrievable after page refresh
PMS-F-014	Offline mode	TC-OFF-01	Create tasks without Supabase configuration	Tasks persisted in LocalStorage; survive browser refresh
PMS-F-015	Settings UI	TC-SET-01	Open settings modal and enter Supabase URL and key	Configuration saved; data service switches to cloud mode
PMS-F-016	Priority assignment	TC-TASK-02	Set task priority to Critical	Priority badge displays correctly on task card
PMS-F-017	Workload balance	TC-WORK-01	Assign tasks to multiple team members	Dashboard shows per-member point distribution
PMS-F-018	Documentation viewer	TC-DOCS-01	Navigate to Docs view	Documentation content renders correctly

13.2 Non-Functional Requirements Traceability

Req ID	Requirement	Test Case ID	Test Description	Pass Criteria
PMS-NF-001	Sub-50ms transitions	TC-PERF-01	Measure drag-and-drop operation latency via Performance API	Operation completes in under 50ms
PMS-NF-002	99.9% availability	TC-AVAIL-01	Verify offline mode activates seamlessly on network disconnection	Application remains functional with no user-visible errors
PMS-NF-003	Cross-browser support	TC-COMPAT-01	Execute core workflows on Chrome, Firefox, and Edge	All features render and function identically
PMS-NF-004	TLS encryption	TC-SEC-01	Inspect network requests during Supabase API calls	All requests use HTTPS with TLS 1.3
PMS-NF-005	100+ task capacity	TC-LOAD-01	Create 100 task records and verify UI responsiveness	Application renders without lag; all tasks accessible
PMS-NF-006	Network recovery	TC-RECOV-01	Disconnect network, create tasks, reconnect	Offline tasks preserved; no data loss

14. Test Metrics and Reporting

14.1 Metrics Collected

Metric	Definition	Target
Test Case Execution Rate	Percentage of planned test cases executed	100%
Test Pass Rate	Percentage of executed test cases that passed	95% or higher
Defect Density	Number of defects per module	Less than 2 per module
Defect Aging	Average time from defect discovery to resolution	Less than 24 hours
Requirement Coverage	Percentage of SRS requirements with at least one mapped test case	100%
Code Coverage (Unit Tests)	Percentage of utility module code exercised by Vitest	80% or higher

Metric	Definition	Target
CI/CD Pass Rate	Percentage of GitHub Actions workflow runs that pass	95% or higher

14.2 Reporting

Report	Frequency	Audience
Daily Test Execution Status	During active test execution phase	Team Lead, QA Lead
Defect Triage Report	Per defect discovery	Developer assigned to fix
CI/CD Pipeline Status	Per push/PR (automated)	All team members
Final Test Summary Report	End of testing cycle	Course Evaluators, Team Lead

14.3 Test Summary Report Contents

The final Test Summary Report will include:

- Total test cases planned, executed, passed, failed, and blocked
- Defect summary (total found, fixed, deferred, by severity)
- Requirement coverage matrix showing 100% traceability
- CI/CD pipeline execution history with pass/fail statistics
- Recommendations for future test cycle improvements

15. Approvals

Role	Name	Signature / Date
QA Lead	GURUBELLI YEKAMBAR ESHWAR RAO (PES2UG24CS173)	Approved / 15-09-2026
Test Architect	Darshan P Pawar (PES2UG24CS143)	Approved / 15-09-2026
Course Coordinator	Software Engineering Faculty	Approved / 15-09-2026